

Zuschnitt-Vorhersage-Tool für Floatglas-Slumping

Neutraler Projektpitch für ein kalibriertes Zuschnitt-Assistenztool auf Basis realer Ofenbrände

Projektidee: Aus einer Heightmap der Negativform, der Glasdicke und dokumentierten Testbränden soll ein Python-Tool einen SVG-Zuschnitt vorschlagen, der definierte Formbereiche zuverlässig mit Floatglas bedeckt und kritische 90°-Kanten möglichst exakt einhält.

1. Ausgangspunkt und Ziel

- Das Projekt richtet sich auf das Slumping von 3-4 mm Floatglas auf individuell hergestellten Negativformen aus Gips/Schamotte. **Die Brennkurve bleibt zunächst konstant, ebenso das Formmaterial.** Dadurch wird das Problem auf einen konkreten, kalibrierbaren Werkstattprozess reduziert.
- **Input:** Heightmap der Negativform, Glasdicke, Start-Zuschnitt, Nach-Brand-Maske und Metadaten.
 - **Output:** SVG-Zuschnittsvorschlag mit Auswertung, ob Mussbereiche bedeckt sind und ob verbotener Überstand auftritt.
 - **Nutzen:** Der erste Mehrwert entsteht bereits, wenn das Tool zuverlässiger ist als 'nach Gefühl zuschneiden' und mit jedem Brand bessere lokale Korrekturen vorschlägt.

2. Technische Grundidee

Die STL-Negativform wird außerhalb des Python-Tools mit C++/PicoGK in eine Heightmap überführt. Diese Heightmap wird als .npy-Datei in den Datenordner des Python-Programms gelegt. Aus ihr erzeugt das Programm automatisch Ziel- und Verbotsbereiche für die Zuschnittbewertung.

Heightmap-Bereich	Interpretation im Tool
$z < 0 \text{ mm}$	Form geht nach unten: dieser Bereich muss nach dem Slumping mit Glas bedeckt sein.
abrupter Höhengsprung $> \text{ca. } 4 \text{ mm}$	Kritische 90°-Kante: bis zur Kante muss Glas liegen, hinter der Kante darf kein Glas liegen.
Kantenband, z. B. $\pm 1\text{-}2 \text{ mm}$	Toleranzzone für die Sollkante, damit die Bewertung numerisch stabil bleibt.
$z \approx 0 \text{ mm}$ ohne harte Kante	Rand-/Auflagebereich: je nach Form kann hier Überstand erlaubt oder neutral bewertet werden.

3. Aufbau eines Datenelements pro Testbrand

Ein Testbrand besteht aus wenigen, sauberen Dateien mit identischem Maßstab, Ursprung und Raster. Für die erste Version reichen binäre Schwarz/Weiß-Masken und eine Heightmap in Millimetern.

Datei	Inhalt
form_heightmap.npy	Heightmap der Negativform in mm; gleiche Auflösung und gleicher Ursprung wie die Masken.
pre_burn_mask.png	Binäre Maske des Start-Zuschnitts vor dem Brand. Schwarz/weiß eindeutig, kein JPG.
post_burn_mask.png	Binäre Maske der finalen Glasfläche nach dem Brand, aus entzerrtem Topfoto oder manuell nachgezeichneter Kontur.

metadata.json	Glasdicke, Brennkurven-ID, Form-ID, Pixelgröße, Testnummer, Datum, Notizen zu Blasen/Kontakt/Überstand.
blank.svg	Geplanter Zuschnitt als SVG; das Tool gibt neue Vorschläge ebenfalls als SVG aus.

Fotodaten sollten über Referenzmarker entzerrt werden. Entscheidend ist, dass Startmaske, Endmaske und Heightmap pixelgenau im selben Koordinatensystem liegen.

4. Verarbeitung im Python-Tool

- **1. Laden:** Heightmap, Masken und Metadaten werden eingelesen.
- **2. Regeln ableiten:** Aus der Heightmap entstehen must_cover, edge_band und forbidden Bereiche.
- **3. Brand auswerten:** Das Tool misst Bedeckung, fehlende Fläche, verbotenen Überstand, minimale Randreserve und Kantenabweichung.
- **4. Kalibrieren:** Start- und Endmaske werden verglichen; daraus werden Korrekturwerte für 3 mm und 4 mm Glas abgeleitet.
- **5. Vorschlagen:** Für eine neue Heightmap erzeugt das Tool einen SVG-Zuschnitt mit lokalen Reserven und Rücknahmen an 90°-Kanten.

Kennzahl	Bedeutung
coverage_target_%	Wie viel Prozent der Muss-Zone ist bedeckt?
missing_area_mm2	Wo und wie viel Glas fehlt?
forbidden_area_mm2	Wie viel Glas liegt in verbotenen Bereichen?
edge_error_mm	Wie weit liegt die finale Glaskante von der Sollkante entfernt?
min_margin_mm	Kleinste gemessene Sicherheitsreserve in erlaubten Randbereichen.

5. Schlanke technische Architektur

Das MVP sollte als Python-Projekt aufgebaut werden. C++/PicoGK bleibt die vorgelagerte Geometrie-Stufe für die Heightmap-Erzeugung.

Bereich	Python-Libraries
Arrays / Heightmaps	numpy, scipy
Bildmasken / Entzerrung	opencv-python, scikit-image
2D-Geometrie	shapely
SVG-Export	svgwrite
Debug-Visualisierung	matplotlib
Metadaten / Tabellen	json / pydantic, pandas

Grobe Modulstruktur: data_loader.py, constraints_from_heightmap.py, evaluate_burn.py, calibrate_parameters.py, suggest_svg_blank.py, report_debug.py.

6. Sinnvoller Umfang an Testschmelzungen

Für den Anfang zählt Datenqualität stärker als Datenmenge. Die ersten Tests sollten möglichst sauber dokumentiert und nur wenige Parameter gleichzeitig verändern.

Stufe	Sinnvoller Umfang
Erste Demo	9 vorhandene Tests plus 4-6 neue, sauber dokumentierte Testbrände.
Brauchbare Kalibrierung	Ca. 12-18 verwertbare Brände pro Glasdicke bei gleicher Brennkurve und gleichem Formmaterial.
Solide Validierung	Ca. 20-30 Brände pro Glasdicke, davon einige Formen bewusst als Validierung zurückhalten.

- Die Testformen sollten verschiedene Tiefen, Radien, Steigungen und 90°-Kanten enthalten.
- 3 mm und 4 mm Glas sollten getrennt ausgewertet werden, weil sie bei gleicher Brennkurve unterschiedlich reagieren.
- Für jeden Brand sollte mindestens ein entzerrtes Topfoto nach dem Brand und eine saubere Endmaske existieren.

7. Entwicklungsaufwand

Umsetzung	Grobe Dauer
MVP als Python-CLI mit Masken, Kennzahlen und einfachem SVG-Vorschlag	ca. 50-90 Stunden
Ordentlicher Prototyp mit Debug-Bildern, Kalibrierparametern und stabiler Datenstruktur	ca. 120-220 Stunden
Robuster Werkstatt-Assistent mit Fotoentzerrung, Validierung und besserer Bedienung	ca. 200-350 Stunden

8. Ausblick

Das Projekt kann sehr schlank starten: Heightmap rein, Testdaten einlesen, SVG-Zuschnitt vorschlagen, Brand auswerten, Parameter verbessern. Später kann die Kalibrierung durch einfache Modelle wie lineare Regression, nearest-neighbor-Vergleiche ähnlicher Formbereiche oder Gaussian Processes erweitert werden.

Langfristige Motivation: Ein Physics-Informed Neural Network, kalibriert auf einen konkreten Glasofen, könnte als Assistent helfen, Slumping-Versuche gezielter zu planen und weniger Brände durch reines Ausprobieren zu verlieren.